

Flying Hirsch - Drone-based Wildlife Detection & Movement Analysis

Introduction

Monitoring wild animals from the air is hard: thermal drone footage is low-contrast, the animals are tiny (often just 20–60 px²), they look like blurry blobs and the drone itself keeps moving, so the whole scene drifts between frames.

Flying Hirsch is a computer-vision pipeline that tackles exactly this problem. It takes thermal images recorded by a drone and:

1. **Detects** animals and draws a bounding box around each one,
2. **Tracks** every animal across consecutive frames with a stable, unique ID and
3. **Analyses movement** by reconstructing each animal's trajectory and spatial distribution over time.

The result is a reusable workflow - from raw dataset to annotated detections, tracked IDs and trajectory videos - that turns raw thermal flights into usable wildlife movement data.

Team

- Karim Ortner
- Lorenz Schlosser
- Jonas Welt

Pipeline Overview

The project is organized as a sequential pipeline. Each stage feeds the next and the documentation below follows exactly this order:

1. **Choose & Explore Dataset**
2. **Data Preprocessing & Annotation**
3. **Train Object Detection**
4. **Train Object Tracking**
5. **Evaluate Model & Create Statistics**
6. **Visualize Results**

The implementation lives in the `pipeline/` folder. Scripts are numbered to match the pipeline order (`01_check_dataset.py`, `02_convert_labels.py`, `03_*exploration`, `04_*preprocessing`, `05_train.py`, `06_validate.py`, `07_predict.py`, `08_tracking.py`, `09_*visualization`).

Our Approach

1. Choose & Explore Dataset

We worked with two thermal wildlife datasets over the course of the project.

ALFS dataset (initial) - zenodo.org/records/18772136

- **Pro:** animals were annotated *by species*.
- **Con:** very blurry images, weird/inconsistent shapes and a huge class imbalance.

Matched Frames Subset of the BAMBI UAV Wildlife Dataset (final) - zenodo.org/records/19034999

- **Pro:** much better image quality, matched frames and ready-made train/val/test splits.
- **Con:** fewer labels and **no species information**.

Because the species labels in ALFS were unreliable and heavily imbalanced, we ultimately collapsed everything into a **single animal class**. Also we used the second dataset instead since the images had a much better quality.

Key dataset facts (BAMBI subset):

Property	Value
Train / Test / Val	~11,391 / ~1,934 / ~840 images
Image size	1024 × 1024
Min / Avg / Max animal area	21 px ² / 1,922 px ² / 61,620 px ²

Scripts used in this stage

- `01_check_dataset.py` - sanity check that every image has a matching label file and reports how many are missing.
- `02_convert_labels.py` - remaps the original 75 fine-grained species IDs down to coarser classes (and finally to a single class), rewriting the YOLO `.txt` labels in place.
- `03_exploration_ALFS.ipynb` / `03_exploration.ipynb` - notebooks that explore label distributions, box sizes and image quality for each dataset.
- `utils/copy_data_oneClass.py` - reproduces the *exact same* train/val/test split from the old ALFS dataset on the new BAMBI data, so results stay comparable across dataset switches.

2. Data Preprocessing & Annotation

Raw thermal frames are noisy, so we clean and normalize them before training. The core helpers live in `utils/preprocess_methods.py` and are driven from `04_preprocessing.ipynb` (with `04b_exploration_annotated.ipynb` to inspect the result).

Cleaning - frames and labels are dropped when they are not usable:

- **Blurry images** are detected via the variance of the Laplacian (low variance = blurry).
- **Mostly-black images** are removed when over ~93% of pixels are near-zero.
- **Invalid / tiny boxes** (out-of-range coordinates, or boxes smaller than ~1.5% of the image) are filtered out.

Normalization - `normalize_thermal()` applies min-max normalization to stretch each image to the full 0–255 range. This boosts contrast and makes faint animals far more visible.

Splitting - We used a utils file (copy_data_oneClass.py) to initialize the split by copying the original split, but the original split included too little images in the val set. Therefore we **corrected the split based on flight ID**, so frames from one flight stay together and we don't have any data leakage.

Re-annotation with Roboflow - the original labels were not sufficient for a good accuracy of the model, so we manually re-annotated **~9,338 images** (train + val) in Roboflow: removed wrong boxes, corrected sizes and added missing animals. We used the matching RGB images during annotation for better insight into what each thermal blob actually was.

3. Train Object Detection

Goal: reliably detect animals in low-contrast thermal drone images.

We use **Ultralytics YOLO** (YOLO26) as the detector. Initially we also tested everything with YOLO11. All shared settings live in `config.py` and `05_train.py` runs the actual training.

Configuration - `config.py`

`config.py` centralizes everything so the other scripts stay clean. The key idea is three **training profiles** (**fast**, **balanced**, **quality**) that trade speed against accuracy by swapping the model size, image resolution and augmentation strength:

```
PROFILE = "balanced" # "fast" | "balanced" | "quality"

PROFILES = {
    "fast": {
        "weights": "yolo26s.pt", "imgsz": 640, "batch": 16, "epochs": 80, ...
    },
    "balanced": {
        "weights": "yolo26m.pt", "imgsz": 1024, "batch": 4, "epochs": 120, ...
    },
    "quality": {
        "weights": "yolo26l.pt", "imgsz": 1024, "batch": 5, "epochs": 150, ...
    },
}

# Validation at native resolution
VAL_IMG_SZ = 1024
VAL_BATCH = 8

# SAHI tiled inference - tuned for ~15-40 px animals in 1024x1024 thermal frames
SAHI_SLICE_SIZE = 512
SAHI_OVERLAP = 0.25
SAHI_CONF = 0.15
SAHI_USE_CLAHE = True

DATA_YAML = "data/alfs.yaml"
TEST_SOURCE = "datasets/annotated/images/test"
BEST_WEIGHTS = "best_weights/best.pt"
MODEL_NAME = "yolo26s_annotated"
```

The `SAHI_*` values configure **sliced inference** (used in prediction): each large frame is cut into overlapping tiles so tiny animals stay large enough to detect. The helper functions `run_name()` and `best_weights()` build consistent run/output names from the active profile.

Training script - `05_train.py`

`build_train_args()` returns the full set of YOLO training arguments. The most important choices for thermal data are highlighted:

For reproduction purpose - we replaced the three profiles with a hard-coded version containing the optimized parameters.

```
def build_train_args(name_addition: str | None = None):
    return dict(
        data="data/alfs.yaml",
        epochs=100, patience=20,
        imgsz=1024, batch=4,
        device=0, workers=8, amp=True,

        optimizer="MuSGD",
        lr0=0.0050, lrf=0.1, weight_decay=0.0005,
        momentum=0.937, warmup_epochs=3, warmup_momentum=0.8,

        # Augmentation - make the model robust to drone viewpoint changes
        mosaic=0.8, close_mosaic=15, copy_paste=0.05,
        scale=0.5, degrees=20, translate=0.1,

        # Thermal: disable hue/saturation jitter (no useful color), keep value
        jitter
        hsv_h=0.0, hsv_s=0.0, hsv_v=0.5,

        val=True, plots=True, verbose=True,
        save_period=10, project="train",
        name=MODEL_NAME + (f"_{name_addition}" if name_addition else ""),
        save=True,
    )
```

Why these settings?

- **hsv_h=0, hsv_s=0** - thermal images carry no meaningful color, so hue/saturation augmentation would only add noise. Only brightness (**hsv_v**) is jittered.
- **Mosaic / copy-paste / scale / degrees / translate** - strong geometric augmentation so the model generalizes across different drone heights, angles and positions.
- **MuSGD optimizer, warmup, cosine-style LR** - stable convergence on a small-object, low-signal dataset.

`main()` supports the three training strategies we experimented with (only one is active at a time):

```
# DEFAULT: train from pretrained COCO weights (yolo26s/m/l.pt)
model = YOLO("yolo26l.pt")
```

```

model.train(**build_train_args())

# CONTINUE: fine-tune from our own best weights
model = YOLO(BEST_WEIGHTS)
model.train(**build_train_args("phase2"))

# ADAPT: convert the first conv layer from 3 channels to 1 (true single-channel
thermal)
first = model.model.model[0]
new_conv = nn.Conv2d(1, first.conv.out_channels, first.conv.kernel_size,
                     first.conv.stride, first.conv.padding,
                     bias=first.conv.bias is not None)
with torch.no_grad():
    new_conv.weight.copy_(first.conv.weight.mean(dim=1, keepdim=True)) # average
    RGB weights
model.model.model[0].conv = new_conv
model.train(**build_train_args())

```

The single-channel adaptation averages the pretrained RGB weights into one channel so we keep the benefit of COCO pretraining while feeding the network true grayscale thermal input.

Training journey

Reaching a usable model took several iterations. Each change moved the mAP@0.5 forward:

Stage	Change	mAP@0.5
1	Raw dataset, no preprocessing, YOLO11	~2%
2	Switched to YOLO26	~3.5%
3	Preprocess + remove blurry images	~6%
4	Limit to classes with enough data (12 → 8)	~8%
5	New dataset, single class (animal / no animal)	~35%
6	Improved annotations for ~9k samples (Roboflow)	~66%
7	Fine-tuned YOLO params + image size 1024	~70%

The two biggest jumps came from (5) simplifying to a single robust class on a cleaner dataset and (6) fixing the annotations.

4. Train Object Tracking

Goal: follow each detected animal across a thermal image sequence with a persistent ID - despite the drone constantly moving. Implemented in `08_tracking.py`.

Because the camera moves, naive tracking would confuse camera motion with animal motion. The pipeline solves this in stages:

1. **Stabilize to a reference frame** - one frame is chosen as the reference. Every other frame is aligned to it using **ORB features (5000)** + a partial-affine transform estimated with RANSAC. This cancels out the drone's movement so the background stays still and only the animals move.
2. **Smooth the transforms** - the per-frame transforms are decomposed (translation, rotation, scale) and median-smoothed over a small window to remove jitter. A neighbor-fallback handles frames where matching fails.
3. **Detect + track** - the trained YOLO detector runs on the stabilized frames and **BoT-SORT** assigns persistent IDs across frames.
4. **Build trajectories** - each animal's path is reconstructed from its bounding-box center points and exported to `trajectories.json` / `trajectories.txt`, alongside overlay images and videos.

This stabilize-then-track design means the recorded trajectories reflect *real animal movement*, not camera drift.

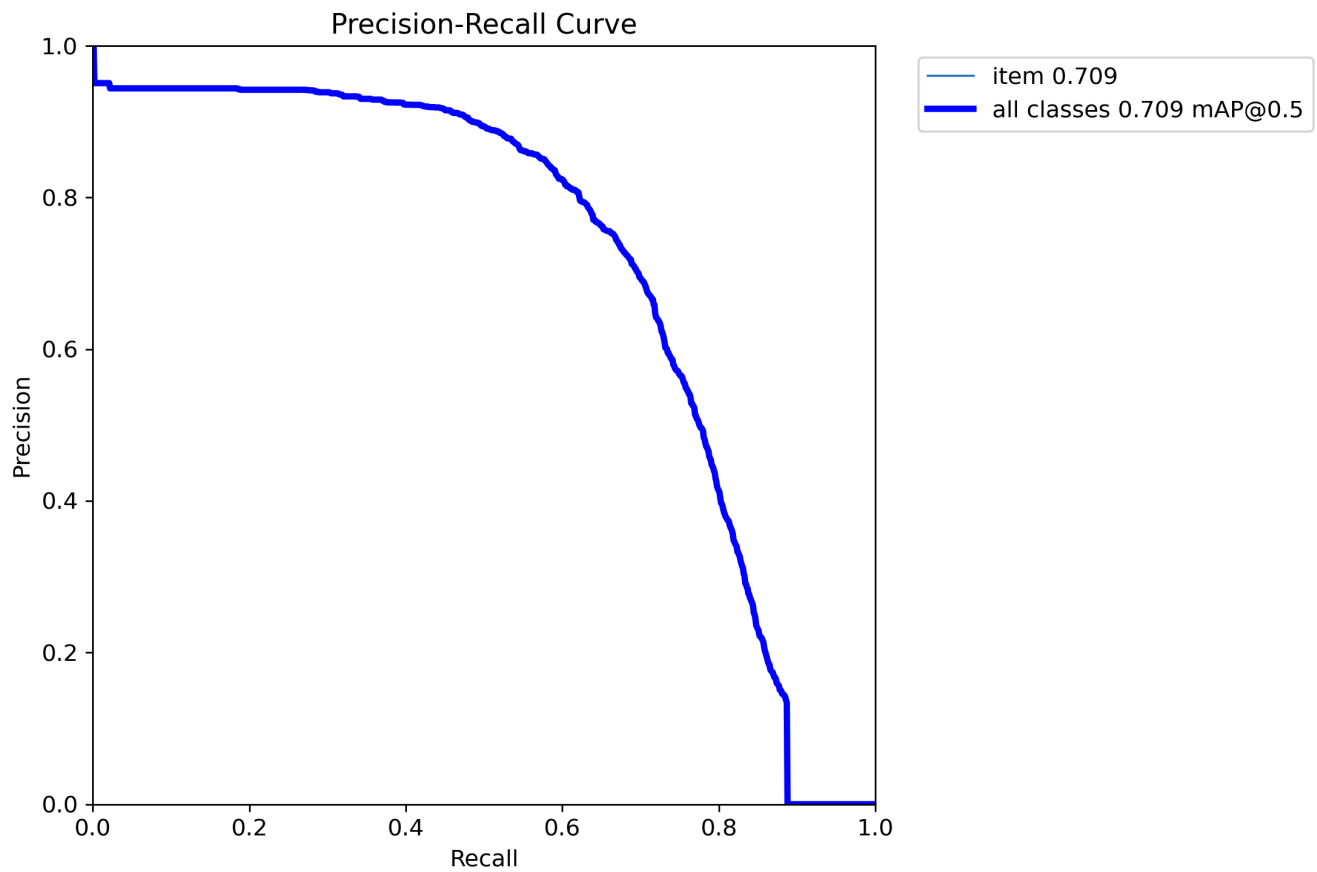
5. Evaluate Model & Create Statistics

`06_validate.py` runs YOLO's validation on the held-out split using our best weights and writes metrics + plots to `runs/detect/validate/`.

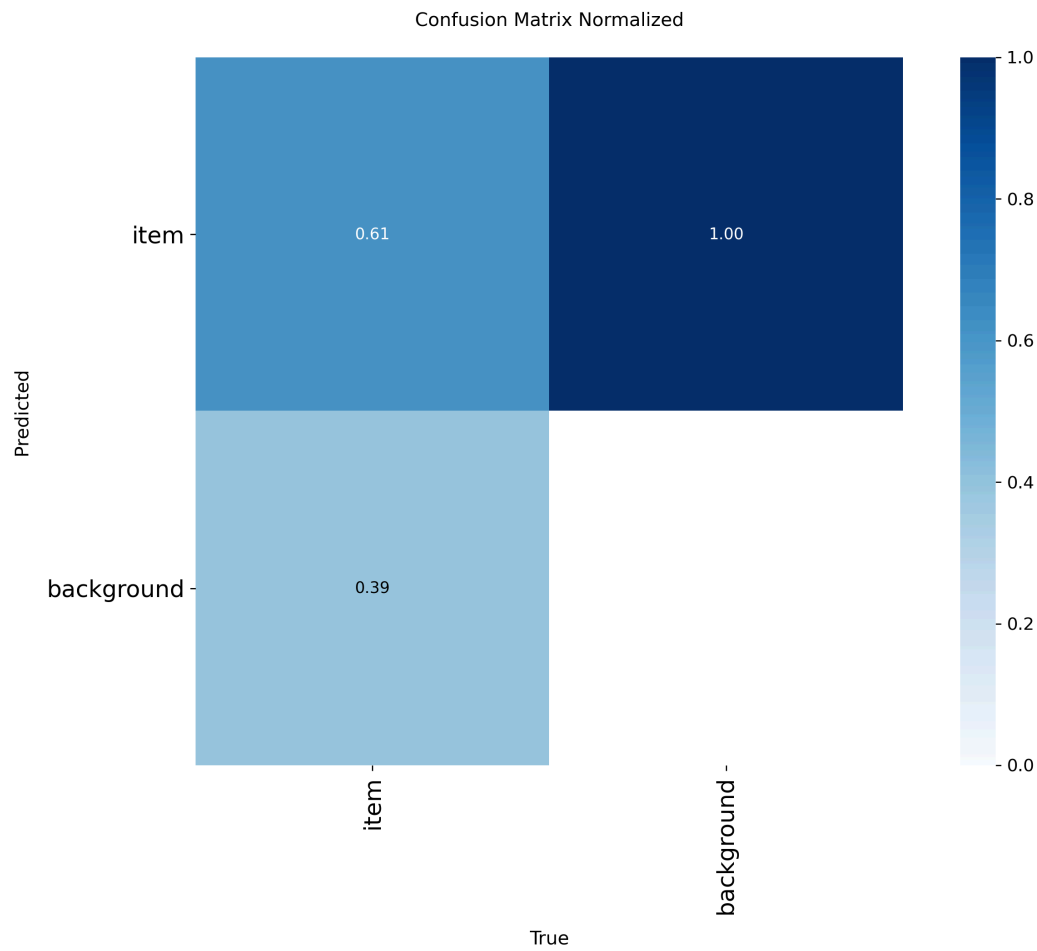
Best model performance:

Metric	Value
Precision	0.760
Recall	0.652
mAP@0.5	0.709
mAP@0.5:0.95	0.247

The **precision-recall curve** confirms strong precision (~0.94) that holds up to ~0.5 recall before falling off - expected behaviour given how small and faint many animals are:



The **normalized confusion matrix** shows the core trade-off: when the model fires, it is usually right, but a fraction of real animals are still missed as background - the hardest part of tiny-thermal-object detection.

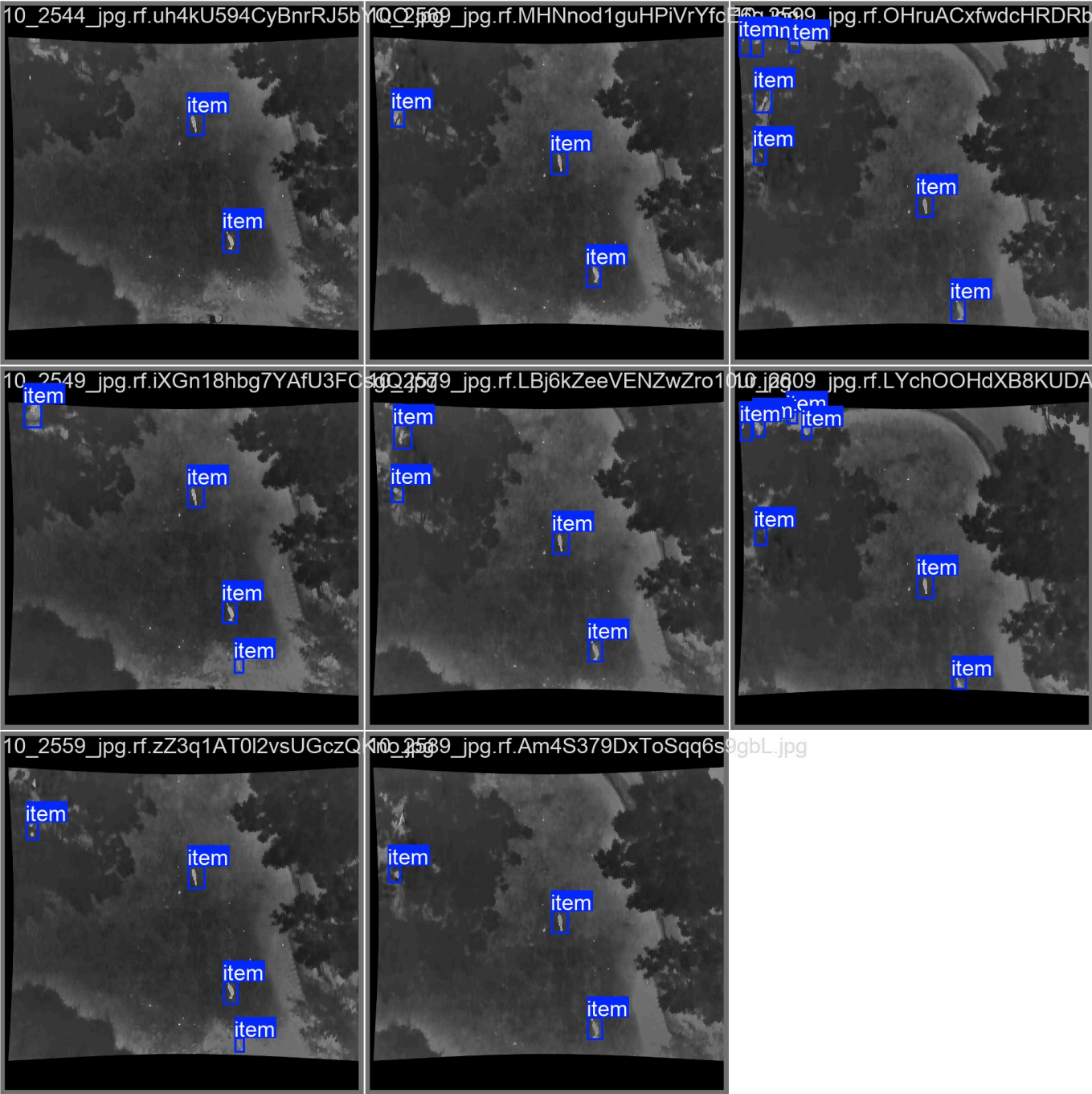


6. Visualize Results

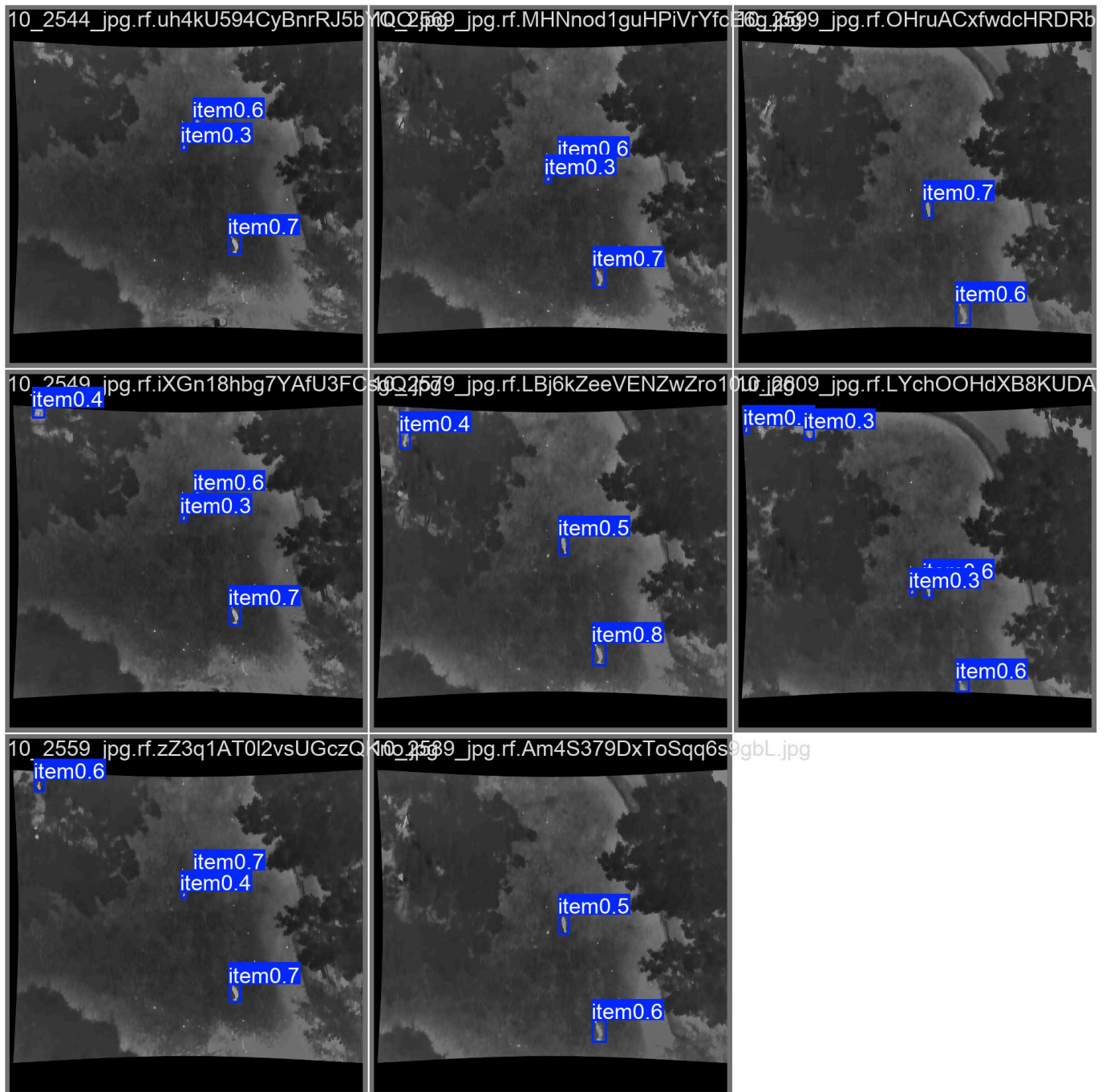
Detection - Ground Truth vs. Prediction

Side-by-side validation batches show the labels (top) against the model's predictions (bottom). The model finds most animals, with confidences shown per box.

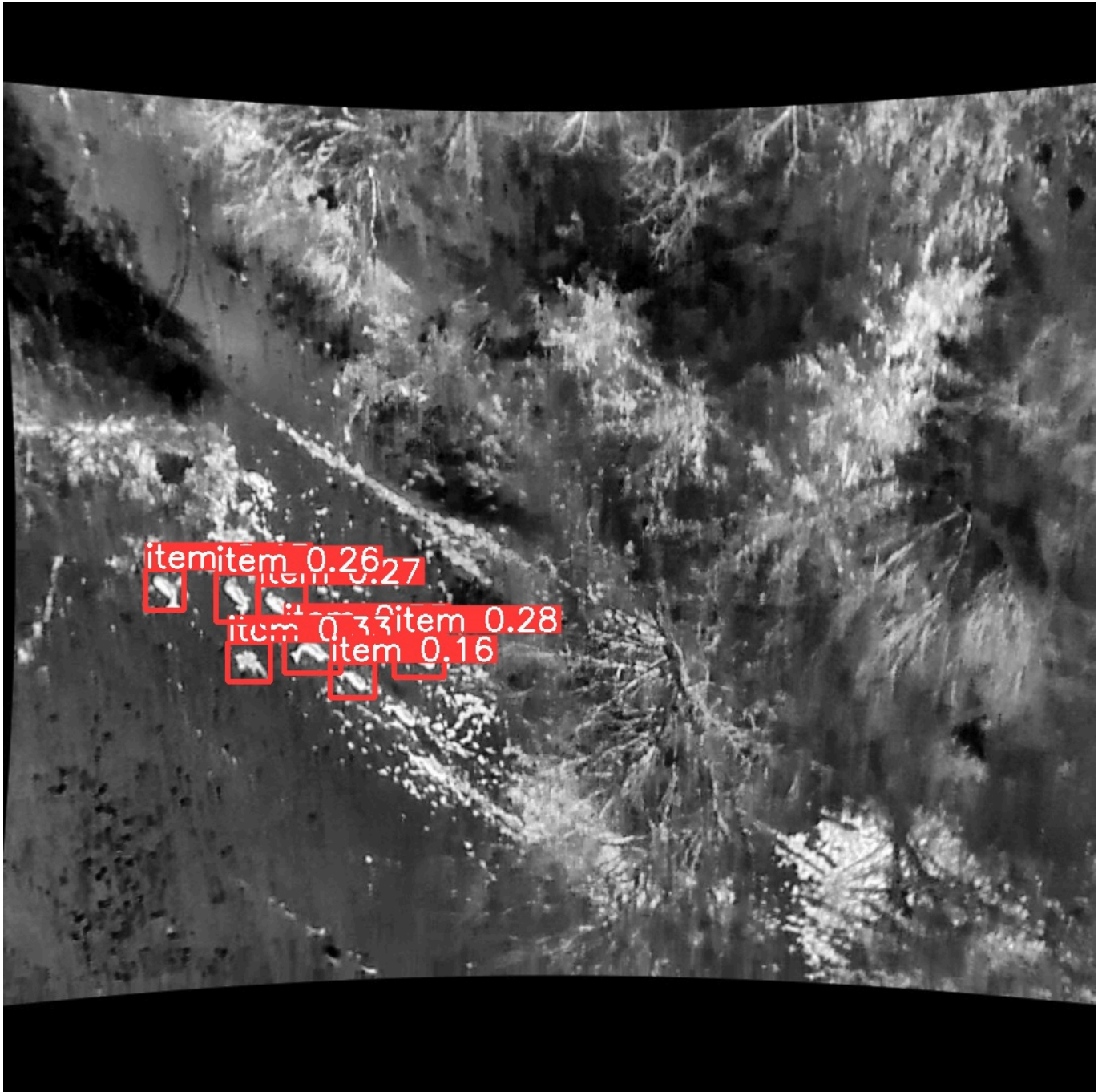
Ground truth:



Prediction:

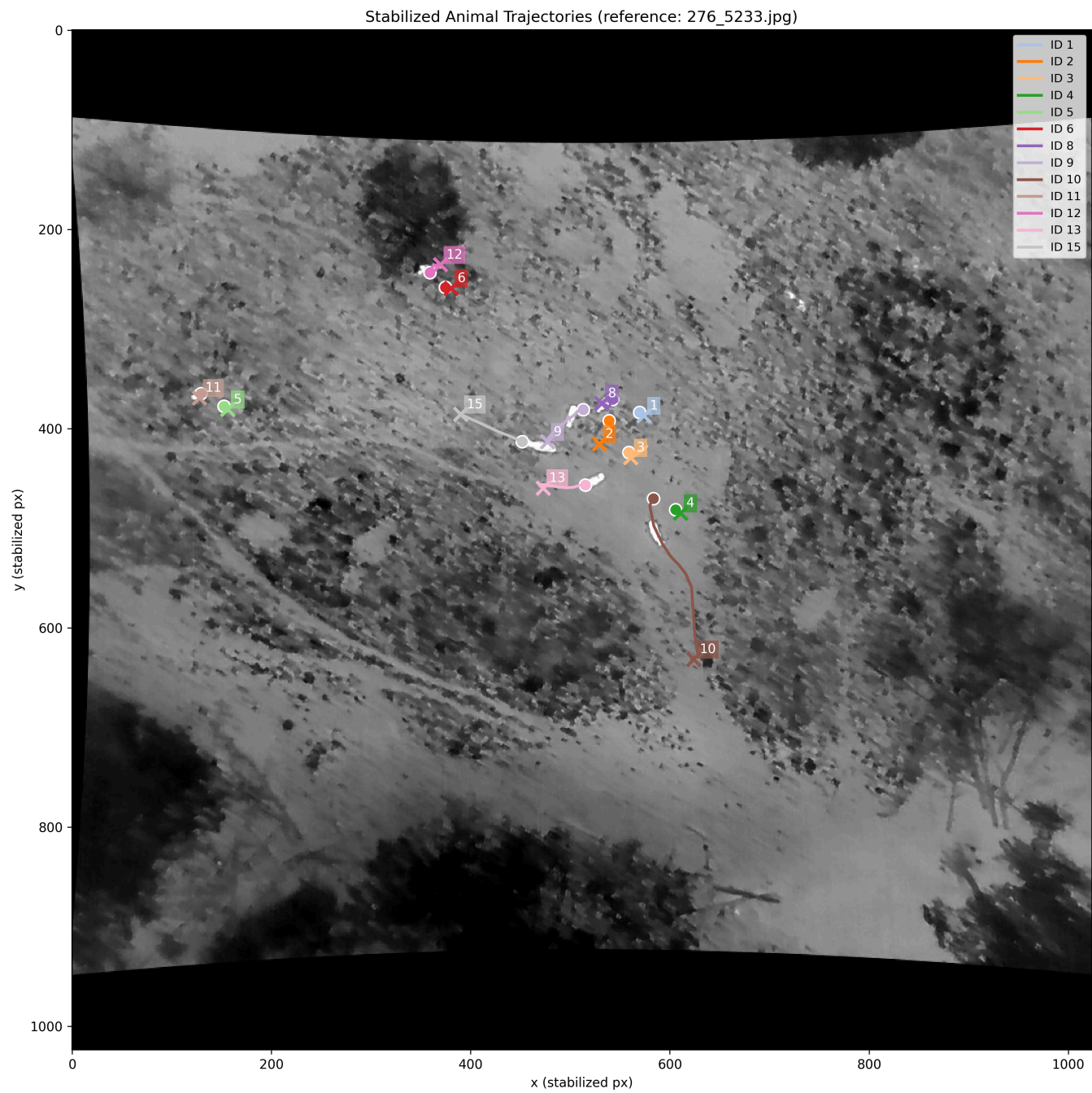


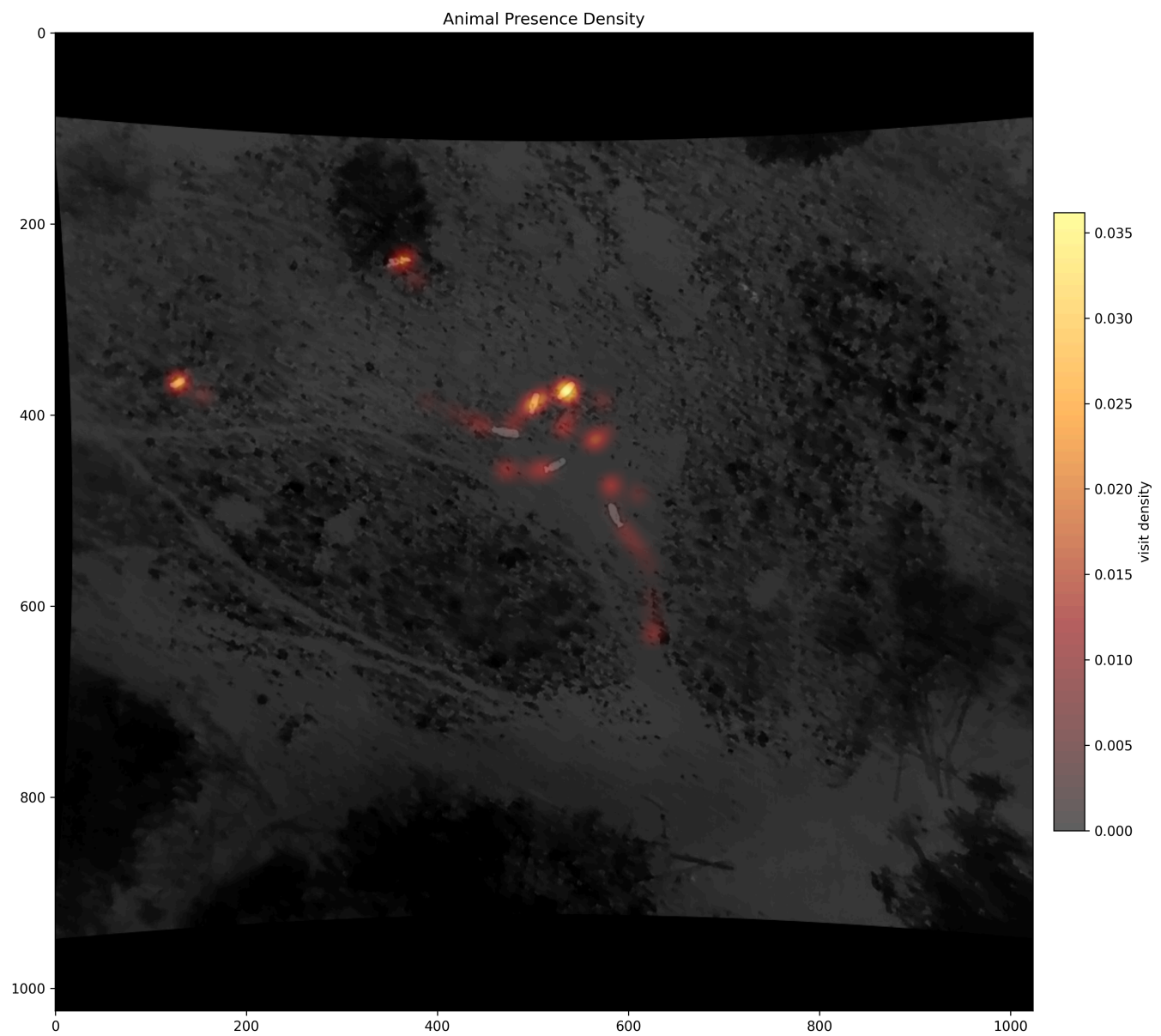
`07_predict.py` additionally runs **SAHI sliced inference** with optional CLAHE contrast boost on the test images, which helps recover very small or faint animals that a single full-frame pass would miss:

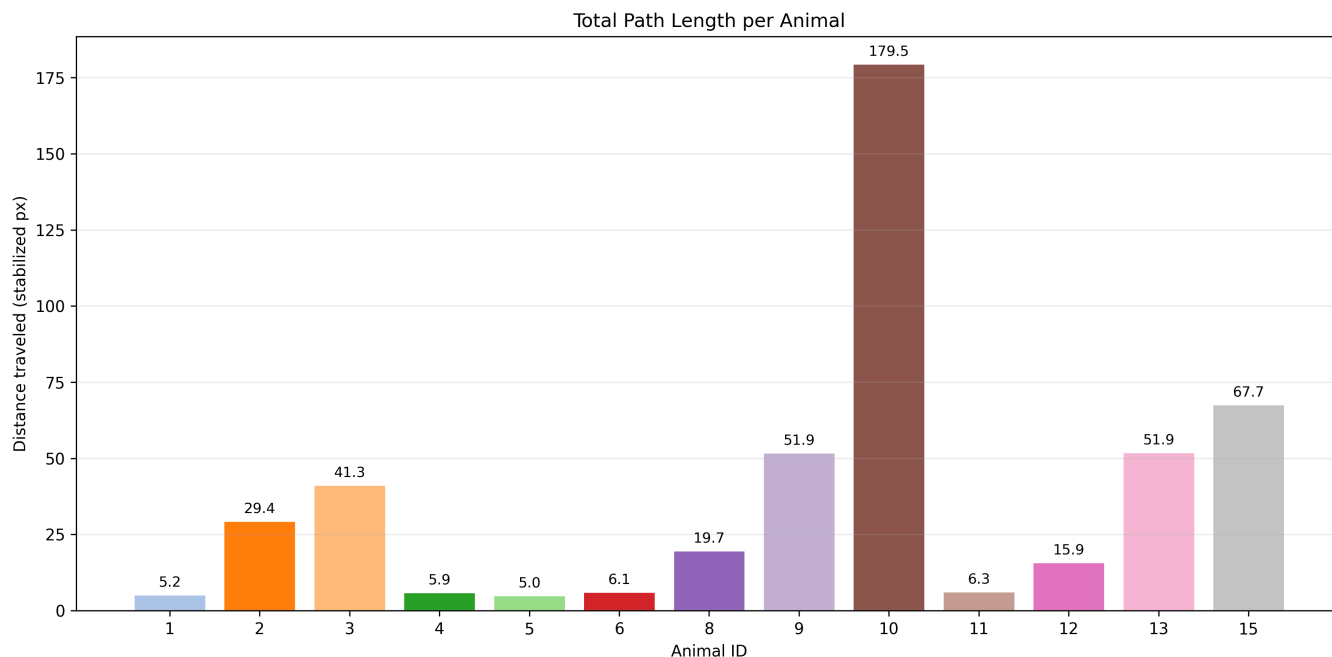


Tracking - IDs & Trajectories

The tracking overlays draw each animal's bounding box, ID and a fading trail of its recent path on the stabilized frames. Below, one sequences ([track_276](#)) show stable IDs and clean trajectory trails after stabilization, a heatmap and also the total distance traveled per ID.







Full results - including the stabilized frames, overlay videos (`tracking_overlay_video.mp4`), trajectory videos and the `trajectories.json` / `trajectories.txt` exports - are saved per sequence under `runs/detect/track/`. The notebooks `09_10_visualizeTracking.ipynb` and `09_276_visualizeTracking.ipynb` render these trajectories per flight.

Summary

Flying Hirsch is an end-to-end thermal wildlife pipeline that goes from raw drone footage to tracked animal trajectories:

- **Data:** moved from the species-labeled but noisy ALFS dataset to the cleaner BAMBI subset, then re-annotated ~9k frames and collapsed to a single robust `animal` class.
- **Detection:** a YOLO26 model, fine-tuned with thermal-specific augmentation and sliced (SAHI) inference, reaches **~0.71 mAP@0.5** - up from ~2% in the first naive attempt.
- **Tracking:** frames are stabilized to a reference (ORB + affine), then YOLO + BoT-SORT produces persistent IDs and per-animal trajectories that reflect real movement rather than camera drift.
- **Output:** annotated detections, ID overlays, trajectory videos and structured trajectory data ready for movement and spatial-distribution analysis.

The biggest lessons: **data quality beats model size** (annotation fixes and a single clean class drove the largest gains) and **stabilization is essential** before any meaningful movement analysis on drone footage.